Engram

AI Memory Management System

Software Design Document

Version 1.0 — April 2026

Shallow-biased retrieval · Filesystem-authoritative storage · Unified orchestrator · KG-as-filesystem proxy · Event-sourced ingest with outbox · Hierarchical consolidation

1. Introduction

1.1 Purpose

This Software Design Document specifies the complete architecture, data model, runtime behaviour, and training procedures for Engram, an AI Memory Management System. Engram gives AI agents and conversational systems persistent, structured long-term memory through a retrieval cascade that enforces the Minimal Sufficient Context (MSC) principle: the context delivered to the frontier LLM should contain exactly enough information to answer the query correctly, and no more.

The document is self-contained. It defines all components, contracts between them, storage responsibilities, failure modes, and the training procedure for the local models. An engineer should be able to implement Engram from this document alone.

1.2 Goals and Non-Goals

1.2.1 Goals

- Provide persistent long-term memory for agents across sessions, with durable storage and deterministic recovery semantics.
- Minimize tokens delivered to the frontier LLM while preserving answer quality (MSC principle).
- Offer a single, coherent memory surface that is navigable both as a knowledge graph and as a filesystem.
- Be modular: the frontier LLM, the Core Model, and the Gating Model are each swappable without architectural changes.
- Be crash-safe: an abrupt process or machine failure must not produce silent data corruption.

1.2.2 Non-Goals (Phase 1)

- Cost-aware query routing across multiple frontier tiers. Deferred. Phase 1 uses one configured frontier LLM.
- Multi-region replication, HA clustering, or automatic failover. Phase 1 runs a single node per storage backend with documented backup/restore procedures.
- Public multi-tenant deployment. Phase 1 assumes a trusted deployer with a single API-key scope; see §12 on auth.

1.3 Design Principles

- **Minimal Sufficient Context (MSC).** Retrieve exactly what is needed. The cascade short-circuits at the shallowest depth that produces sufficient context.

- **Shallow-Biased Depth Prediction.** The Core Model predicts the retrieval depth needed to answer the query, biased toward shallow. If the frontier model is unsatisfied with the delivered context, it requests deeper retrieval. This avoids over-retrieval on the majority of queries without preventing the system from going deep when needed.
- **Configurable Core Model.** The Core Model is an abstraction, not a specific model. It can be a local fine-tuned model (default: Qwen3.5-0.8B), an API model (Claude, GPT-4o-mini), or any model behind a standard completion interface. Swapping the Core Model is a configuration change.
- **Filesystem-Authoritative Storage.** The filesystem is the source of truth for all stored content. The Knowledge Graph (Neo4j) is a derived index carrying topology, embeddings, and semantic metadata. The KG can be rebuilt from the filesystem at any time; the reverse is not true.
- **Event-Sourced Ingest with Outbox.** Every ingest request is recorded in an append-only Event Ledger before any downstream processing begins. All subsequent steps are idempotent and driven off the ledger. A crash at any point leaves the system recoverable without data loss.
- **KG as Filesystem Proxy.** The Knowledge Graph mirrors a hierarchical mem:// filesystem. Every KG node carries a source_uri that locates the original content on disk. Traversing the KG is equivalent to navigating the directory tree.
- **Clear Storage Boundaries.** Operational metadata (processing status, retry counts, task queues, outbox state) lives in SQLite. Semantic metadata (confidence, temporal context, relationships) lives in the KG as node/edge properties. Unstructured content lives on the filesystem. No store crosses into another's responsibility.
- **Retrieval Orchestrator.** A top-level orchestrator owns the multi-step retrieval loop: gate → plan → execute → judge → answer → (optionally) re-enter. The cascade levels (L0 through L4) describe what is retrieved; the orchestrator decides when to stop.
- **Session-Only Caching.** The only caching layer is the session context cache (Redis). There is no KG query cache, no embedding cache, and no application-level LRU cache in Phase 1. Caching layers will be added in later phases when profiling reveals bottlenecks.

1.4 Three-Model Inference Stack

Every inference request in Engram is served by at most three models. No other models are loaded at runtime.

Model	Implementation	Role
Gating Model	BGE-Small-EN-v1.5 (33M params, 384-dim). Frozen encoder. Used only for embeddings in Phase 1;	Generates all embeddings for the KG vector index. Optionally participates in L0 gating

Model	Implementation	Role
	binary classification head attached in the dual-model configuration (see §1.5).	depending on gating configuration.
Gating Classifier (optional)	Separate 33M-parameter classifier (see §1.5). Trained for binary classification over the query surface.	L0 binary gate: Class 0 (bypass retrieval) or Class 1 (needs context). Disabled when retrieval.l0_skip = true.
Core Model	Configurable. Default: Qwen3.5-0.8B local (fine-tuned, see §14). Swappable to any model behind an OpenAI-compatible API.	All intelligence at planning time: write-path gate, S-R-O extraction, depth prediction, retrieval plan generation, coverage judgment, deduplication, entity linking, overview generation, session compaction.
Frontier LLM	Any frontier model (API or local). Claude, GPT-4o, Gemini, etc. Configured once.	Final answer generation. Receives assembled MSC. Emits ANSWER or NEED_MORE verdict through structured output. Has no direct access to the memory system.

1.5 Gating Model — Unified vs Dual Configuration

A key design choice is whether the Gating Model and the embedding generator are the same artifact. Engram supports both configurations; the default is the dual configuration.

Configuration	Description	When to use
Dual (default)	One frozen BGE-Small for embeddings; a separate 33M classifier for L0 gating. The embedding space never changes unless the deployer explicitly chooses to re-embed. The gate can be re-trained or upgraded independently.	Production. Zero risk of embedding-space drift from gate retraining. Simpler migration story. Negligible additional memory cost.
Unified	One BGE-Small with a linear head on the [CLS] token for gating and mean-pooling on the rest for embeddings. Four of six	Experimental. Saves one model load (~130 MB resident). Every gate retraining requires a full re-embed and vector-index

Configuration	Description	When to use
	transformer layers frozen to preserve embedding quality under fine-tuning.	rebuild; only acceptable where that cost is understood and planned for.

Why two modes

The unified configuration is attractive because it saves memory and guarantees that gating and retrieval share a representation space. In practice it couples two optimization cycles that belong apart: gating recall is driven by product experience, while embedding quality is driven by retrieval metrics. Coupling them means every gate improvement triggers a full re-embed migration, which is expensive and risky at scale.

The dual configuration is the safe default. The unified configuration remains supported for constrained deployments and for research comparison.

2. System Architecture

2.1 Architecture Layers

Engram is organized into four layers. Each layer has a narrow responsibility and a defined contract with the layers above and below.

Layer	Components	Responsibility
Interface Layer	REST API (FastAPI), Python SDK	External integration, request validation, authentication, response formatting
Orchestration Layer	Retrieval Orchestrator, Session Manager, Ingest Worker, Consolidation Worker, Reconciliation Worker	Request routing, retrieval loop, session lifecycle, async job execution, crash recovery
Intelligence Layer	Gating Classifier (L0), Core Model (planning + extraction + judgment), Frontier LLM (answer + verdict)	Classification, extraction, query generation, coverage assessment, answer generation

Layer	Components	Responsibility
Storage Layer	Knowledge Graph (Neo4j), Event Ledger + Outbox + Consolidation Queue (SQLite), Session Cache (Redis), Filesystem (mem:// hierarchy)	Persistent state, indexes, write-ahead log, task scheduling, session context, authoritative content

2.2 Storage Architecture

Engram uses three storage backends with strict responsibility boundaries. The filesystem is authoritative for content; Neo4j is a derived index; SQLite is the operational/control-plane store.

Backend	Stores	Rationale
Filesystem (mem://)	Authoritative content: every memory's full body (.md files), directory overviews (overview.md), directory tables-of-contents (.manifest files). All content is addressed by mem:// URIs.	Content consumed as text by models. Filesystem hierarchy mirrors KG structure. Filenames encode semantic summaries. Disaster recovery is straightforward: back up the filesystem.
Neo4j	Knowledge Graph topology: nodes with l0_abstract and l0_embedding, edges (CONTAINS, IS_PART_OF, RELATES_TO, SUPERSEDES, REFERENCES), native vector index on l0_embedding, full-text index on l0_abstract, semantic metadata.	Derived index rebuildable from the filesystem. Holds only what must be queried via Cypher, traversed relationally, or searched via vector similarity.
SQLite (WAL mode)	Event Ledger (append-only log of ingest requests), Outbox tables (fs_outbox, kg_outbox) for crash-safe stepwise processing, Consolidation Task Queue, processing metadata (retry counts, idempotency keys, status).	All operational/write-heavy control data. WAL mode provides concurrent reads during writes. The reconciliation worker drains the ledger and outboxes at-least-once.

Boundary rule
If data is used to filter, sort, or relationally join, it lives as a KG property. If a model consumes it as natural-language context, it lives on disk. If it tracks processing state, it lives in SQLite. Every piece of data has exactly one authoritative home.

2.3 Trust and Authoritativeness

A single rule governs data-flow recovery: the filesystem is authoritative. If the filesystem and Neo4j disagree, the filesystem wins. Concretely:

- The filesystem carries the canonical copy of all content and, via YAML frontmatter on every .md file, the canonical copy of per-node metadata (id, node_type, status, created_at, source_session_id, schema_version).
- Neo4j stores derived indexes: l0_abstract, l0_embedding, edges, and semantic metadata surfaced from frontmatter. A rebuild operation (memcascade rebuild-kg --from {data_dir}) walks the filesystem and recreates Neo4j state from scratch.
- SQLite stores control-plane state only (events, outboxes, consolidation queue). It is not authoritative for any memory content; on loss, pending work is lost but committed memories are preserved on disk.

2.4 High-Level Request Flow

2.4.1 Query flow

1. Client POSTs to /api/v1/query with session_id and query text.
2. Retrieval Orchestrator runs L0 gate (if enabled).
3. If gate returns BYPASS: Frontier LLM is called directly; answer returned.
4. Otherwise Core Model produces an L1 plan: {predicted_depth, mode, entry_points, vector_queries, commands[]}.
5. Orchestrator executes retrieval commands up to predicted_depth. (Orchestrator is not a specific model per se, but a module to run the iterative retrieval and judgment process, involves both core model and frontier LLM.)
6. Orchestrator assembles MSC and calls Frontier LLM.
7. If Frontier LLM returns NEED_MORE: re-enter at predicted_depth + 1 with suggested queries, up to max_reentries.
8. Final answer and retrieval metadata returned to Frontier LLM.

2.4.2 Ingest flow

9. Client POSTs a user/assistant message pair (or turn group) to /api/v1/ingest.

10. Ingest endpoint performs an atomic SQLite INSERT into the Event Ledger with an idempotency key. Returns 200 OK immediately with the event_id.
11. Ingest Worker picks up pending events. For each event it runs: write-path gate → S-R-O + L0 abstract → filesystem write → Neo4j index update → enqueue consolidation tasks. Each step is individually idempotent and persists progress in the outbox tables.
12. Consolidation Worker processes overview/manifest regeneration, atomization, normalization, temporalization, and integration tasks asynchronously.
13. Reconciliation Worker periodically scans the Event Ledger and outbox tables for stuck entries and retries them.

3. Retrieval Cascade

The retrieval cascade is the read path of Engram. It is organized along two orthogonal dimensions:

- **Levels (L0 through L4)** describe what is being retrieved — nothing at all (L0), L0 abstracts via vector search (L1), structural neighbours via graph traversal (L2), directory overviews (L3), full documents with multi-hop graph context (L4).
- **Phases (Plan, Execute, Judge)** describe what is being done at each level — the Core Model plans retrieval, the system executes it, and the Orchestrator judges sufficiency (fused into the next level's Plan when the cascade continues).

The Retrieval Orchestrator (§4) owns the loop that walks through levels and phases. Sections 3.1–3.5 define the semantics of each level in isolation; §4 defines how they are composed and when the loop terminates.

3.1 Level L0 — Binary Query Gate

L0 is a binary gate on the raw user query. It classifies the query as self-contained (BYPASS, no retrieval needed) or context-dependent (CONTINUE, proceed to L1).

Attribute	Specification
Input	Raw user query string
Output	BYPASS or CONTINUE
Gating Classifier	33M-parameter classifier (dual configuration) or [CLS]-head on shared BGE-Small (unified). See §1.5 and §14.1.
Classification threshold	0.3 on the Class 1 (needs-context) probability. Biased toward recall — uncertain queries go to L1.

Attribute	Specification
Regex OR-gate	Final decision = model_output OR regex_match on the first sentence. Regex patterns are listed in §3.1.1.
Memory-hit fallback	On BYPASS, run a cheap vector lookup (one Gating Model embedding + Neo4j vector search, ~30 ms). If the top hit has cosine ≥ 0.75 against the query, override to CONTINUE. See §3.1.2.
User override	If retrieval.l0_skip = true in configuration, the gate is disabled and every query proceeds to L1. See §3.1.3.
Latency target	< 15 ms (gating classifier only) / < 50 ms (with memory-hit fallback, on cache-warm Neo4j).
On BYPASS	Skip L1–L4. Send system prompt + user query directly to Frontier LLM. Retrieval metadata records cascade_depth_reached = "L0".
On CONTINUE	Pass query to L1. The Gating Classifier has no further role in this request.

3.1.1 Regex patterns

The regex OR-gate catches surface patterns that a small classifier may miss. Applied to the first sentence of the query only, case-insensitive. Matching any pattern forces CONTINUE regardless of classifier output.

```
# Sentence-initial conjunctions (anaphora to prior turn)
^\s*(but|and|so|or|then|also|actually|wait)\b

# Deictic references to prior conversation
\b(previously|earlier|last turn|as i said|as i mentioned|you mentioned|you said|we
discussed|above|before)\b

# Possessives that typically reference user memory
\bmy
(wife|husband|partner|kids?|children|family|address|birthday|password|schedule|calendar|team|man
ager|boss)\b

# First/second-person references implying prior context
^\s*(he|she|they|it|that|this|these|those)\s

# Syntactic incompleteness (ends with coordinator, dangling prep)
\b(and|or|but|because|since|if|when|while|about|for|with)\s*\?*\s*$
```

These patterns are deployment-tunable. The default set targets conversational assistants; a coding-agent deployment may replace the possessives list with code-specific triggers.

3.1.2 Memory-hit fallback

The L0 gate's failure mode is the linguistically self-contained query that nonetheless requires memory, e.g. "what is my wife's birthday?" — no deixis, no anaphora, but answerable only from long-term memory. Left alone, the Frontier LLM would either hallucinate or admit ignorance.

The memory-hit fallback closes this hole cheaply. Whenever the gate returns BYPASS, the orchestrator runs a single Gating Model embedding of the query and one vector lookup against Neo4j's l0_embedding index. If the top-1 cosine similarity exceeds memory_hit_threshold (default 0.75), the gate's decision is overridden to CONTINUE and the L1 plan is invoked with the vector hit already available.

- Cost on BYPASS: ~20 ms (one embedding + vector lookup). Absorbed in the gating latency budget.
- False-positive cost: L1 proceeds as usual; worst case is an extra Core Model call on a query that didn't need retrieval.
- False-negative cost: the gate still misses queries that need context but have no close lexical match in memory. These remain the responsibility of the specialized edge-case model in §14.1.4.

3.1.3 Configurable L0 skip

Deployments where nearly every query needs context (e.g., personal assistants with rich user memory) can set retrieval.l0_skip = true to bypass L0 entirely. Every query proceeds to L1 planning. The cost is one additional Core Model call (~1–1.5 s) on truly self-contained queries; the benefit is the elimination of gate false-negatives as a class of error.

3.2 Level L1 — Plan & Vector Search

L1 is the planning level and the most consequential Core Model call in the cascade. It takes the user query plus session context and produces a complete retrieval plan covering all levels up to the predicted depth. It also executes the vector search for L1 and records the L0 abstracts that come back.

Attribute	Specification
Model	Core Model
Input	User query + session context + mem:// tree (rendered breadth-first under tree_display_max_tokens, see §7.5) + KG schema summary + L0 memory-hit (if produced by §3.1.2)
Output schema	See §3.2.1 below

Attribute	Specification
Execution	After plan generation, the orchestrator runs any Plan-specified vector queries against the Neo4j vector index on l0_embedding, retrieves top-K per query (default K = 10, capped by max_l1_vector_results = 30), applies MMR-based dedup, and passes the merged L0 abstracts to the next level's Plan.

3.2.1 L1 output schema

```
{
 "session_sufficient": true | false,
 "predicted_depth": "SESSION" | "L2" | "L3" | "L4",
 "mode": "AGFS" | "KG" | "HYBRID",
 "entry_points": ["mem://user/entities/alice/"],  // AGFS/HYBRID only
 "vector_queries": ["reformulated query 1", "..."], // KG/HYBRID only
 "commands": [                                  // ordered, executed by orchestrator
   { "template": "02_overview_for", "params": { "uri": "mem://..." } },
   { "template": "find", "params": { "query": "...", "k": 10 } }
 ],
 "session_answer_context": "..."                // if session_sufficient = true
}
```

The Core Model is instructed to bias predicted_depth toward shallow: it should name the minimum depth it believes will suffice, not the depth that guarantees sufficiency. If the frontier model later disagrees, re-entry (§4.3) handles it.

3.2.2 Mode selection

Mode	When selected
AGFS	The query maps cleanly to a known subtree (e.g., "what are Alice's preferences" → mem://user/entities/alice/preferences/). The plan contains filesystem navigation commands (ls, overview, cat) and no vector search.
KG	The query has no obvious entry point but is likely answerable from a single or few memories found by semantic similarity. The plan contains vector queries and optionally short-hop relational traversal.
HYBRID	The plan contains both entry points and vector queries. This is the default for queries that reference a known entity in combination with an open-ended question. Most queries end up in HYBRID.

3.2.3 Short-circuit at L1

If the Core Model marks session_sufficient = true, it also emits session_answer_context extracted from the session. The orchestrator assembles the MSC from this context alone and proceeds directly to Frontier LLM answer generation. L2–L4 are skipped.

3.3 Level L2 — Graph Traversal over Abstracts

L2 traverses the KG from the L1 entry points or vector hits, following structural and semantic edges. It adds relational context that flat vector search cannot provide.

Attribute	Specification
Entry condition	L1 predicted_depth ≥ L2 AND (L1 vector results insufficient OR L1 plan explicitly requires L2 traversal).
Model	Core Model (Plan phase refines L1 commands based on L1 execution results; Judge phase is fused).
Input	User query + session context + L1 results (L0 abstracts, entry points).
Retrieval vocabulary	Parameterized Cypher templates only (see §7.4). The Core Model selects template + params. No raw Cypher.
Execution	Orchestrator runs selected templates against Neo4j with the read-only role (§12.3). All templates enforce WHERE n.status = 'ACTIVE' by default unless a history-scoped template is chosen.
Judgment	Fused into L3 plan generation: the Core Model at L3 receives L2 results and decides whether to proceed. No separate Judge call.
Key distinction from L1 L1 finds "what is semantically similar to the query" via vector search. L2 finds "what is structurally connected to what L1 found" via graph traversal. L2 never introduces new candidate entities; it only enriches the candidates L1 produced.	

3.4 Level L3 — Directory Overviews

L3 reads overview.md files from the filesystem for the top candidates from L2. It is the first level that consumes non-trivial token budget per memory.

Attribute	Specification
Entry condition	L1 predicted_depth ≥ L3, OR L2 judgment (fused into L3 plan) insufficient.
Input	User query + session context + L2 results.
Retrieval	For top-ranked L2 nodes, follow source_uri and read overview.md from {data_dir}/{source_uri}/overview.md. Core Model may emit

Attribute	Specification
	deeper Cypher template selections (2-hop CONTAINS chains, temporal filters) alongside overview reads.
Output	Overviews for top candidates (up to overview_budget_tokens, default 6000) + deeper KG traversal results.
Judgment	Fused into L4 plan generation, same as §3.3.

3.5 Level L4 — Full Documents & Multi-Hop

L4 reads full L2 documents from the filesystem and performs multi-hop graph traversal for relational reasoning. It is the deepest and most expensive retrieval level.

Attribute	Specification
Entry condition	L1 predicted_depth = L4, OR L3 judgment (fused into L4 plan) insufficient.
Input	User query + session context + L3 results.
Retrieval	Full .md files at source_uri for confirmed high-relevance nodes. Multi-hop Cypher template selections (2–4 hops along CONTAINS and RELATES_TO edges) for relational reasoning.
Budget	full_doc_budget_tokens (default 20,000). When the cumulative budget is exceeded, remaining documents are dropped in favour of overviews.
Termination	L4 always terminates the cascade. After L4 execution, MSC assembly proceeds regardless of coverage judgment. Further re-entry is the Frontier LLM's responsibility (§4.3).

4. Retrieval Orchestrator

4.1 Overview

The Retrieval Orchestrator owns the top-level read loop. It is a discrete, inspectable component — not an emergent behaviour of the cascade. Every query passes through it. Re-entry, re-planning, and budget enforcement all live here.

The orchestrator's responsibilities:

- Invoke L0 gate (if enabled) and honour its decision.
- Invoke L1 planning and receive the retrieval plan.

- Execute plan commands level by level up to predicted_depth, accumulating retrieved context.
- Fuse Plan and Judge phases (§4.2) so that each Core Model call past L1 produces both the next level's commands and the implicit judgment of the previous level.
- Assemble MSC and invoke the Frontier LLM.
- Handle NEED_MORE verdicts from the Frontier LLM up to max_reentries (§4.3).
- Emit retrieval_metadata for telemetry.

4.2 Plan-Judge Fusion

A naive cascade makes separate Core Model calls for Plan at each level and Judge at each level, yielding 2 ·(depth) calls. This is too expensive (a deepest-level L4 query takes 8+ sequential Core Model calls before the Frontier LLM runs).

Engram fuses the Judge phase of level N into the Plan phase of level N+1. The Core Model at L2 plan-time receives L1 results and decides both (a) whether L1 was sufficient, and (b) if not, what L2 commands to run. This reduces the per-level Core Model cost from 2 calls to 1.

Schema for fused plan output at levels ≥ L2:

```
{
 "previous_level_sufficient": true | false,
 "terminate_cascade": true | false,   // if previous level judged sufficient
 "commands": [ ... ],                 // empty if terminate_cascade = true
 "coverage": {
  "aspects_covered": ["who is Alice", "what project"],
  "aspects_missing": ["when did project start"]
 }
}
```

When terminate_cascade = true, the orchestrator skips all remaining levels and assembles MSC immediately.

> **Model-call budget after fusion**
> Happy-path L4 query: L1 plan + L2 plan-with-judge + L3 plan-with-judge + L4 plan-with-judge = 4 Core Model calls, plus 1 Frontier call. Down from 8+1 in the unfused design. With a re-entry the worst case is 5 Core + 2 Frontier.

4.3 Frontier Re-Entry Protocol

After MSC assembly, the orchestrator calls the Frontier LLM with a structured-output requirement. The Frontier LLM MUST emit one of two verdicts:

```
// Success path
{
```

```
  "verdict": "ANSWER",
  "answer": "Alice is working on Project Atlas, a distributed ML pipeline."
}

// Re-entry request
{
  "verdict": "NEED_MORE",
  "reason": "The context describes Alice but does not cover the project timeline.",
  "suggested_queries": ["Project Atlas start date", "Atlas milestones"],
  "suggested_depth": "L3"   // optional hint; orchestrator may override
}
```

4.3.1 Re-entry semantics

- **Trigger.** Verdict = NEED_MORE from the Frontier LLM.
- **Depth escalation.** Re-enter at max(current_depth + 1, suggested_depth). If already at L4, no further depth is possible; the orchestrator calls the Frontier LLM one more time with the full accumulated context and forces a best-effort ANSWER.
- **New queries.** suggested_queries are prepended to the Core Model's next plan input so the retrieval reflects the frontier's specific ask.
- **Maximum re-entries.** max_reentries in configuration (default 2). Every re-entry is one additional L*n* plan-with-judge call plus one Frontier call.
- **User-visible behaviour.** Re-entry is hidden from the user. The final answer is streamed or returned as a single response. retrieval_metadata.reentries records the count for telemetry.

4.3.2 Streaming consideration

Because the verdict field gates whether to stream the answer, streaming is disabled on the first call and enabled only after an ANSWER verdict is observed. For a two-pass query this adds one round-trip of latency on the non-streamed first pass; the answer itself (second pass) streams normally. An engineering optimization available in later phases is to use the frontier provider's tool-use mechanism to branch at verdict detection time without disabling streaming.

4.4 MSC Assembly

After cascade termination, the orchestrator compiles the selected context into a structured prompt. Assembly order matters for attention allocation:

14. System prompt (static). Defines the frontier model's role, output schema, and re-entry protocol.
15. Session context. Most recent turns in raw form; older turns compacted.
16. Retrieved LTM context. Ordered by retrieval level (L1 first, then L2, L3, L4). Each retrieved node is annotated inline.

17. User query. Repeated verbatim at the end for recency bias in attention.

4.4.1 Node annotation requirements

Every retrieved LTM node surfaced to the Frontier LLM must carry inline metadata. Omitting these allows the frontier to confuse superseded facts with current ones, or to treat low-confidence extractions as authoritative.

Annotation	Format
Status	[ACTIVE] | [HISTORICAL: superseded 2026-03-12] | [LOW_CONFIDENCE: 0.45]
Source URI	(mem://user/entities/alice/)
Retrieval level	(L1 vector / L2 graph / L3 overview / L4 full)
Temporal scope	Derived from metadata.temporal.* if present ("as of March 2026")

By default, HISTORICAL nodes are not included in MSC. They are surfaced only when the user query explicitly references history (e.g., "what did I used to believe about X") or when the Core Model explicitly requests a history-scoped Cypher template at L2+.
Low-confidence nodes are included with the [LOW_CONFIDENCE] annotation but never without it.

4.4.2 Token budget

Region	Default share of frontier context window
System prompt + user query	Up to 10%
Session context	Up to 30%
Retrieved LTM context	Up to 50%
Slack (reserved for answer)	10%

Ratios are configurable. When the assembled context exceeds budget, a compression pass (LLMLingua perplexity-based filtering or Core-Model summarisation) reduces it before assembly. The compression pass drops low-confidence nodes first, then compresses session context, then drops lowest-ranked LTM content.

5. Write Path — Ingest Pipeline

5.1 Design Principle: Event-Sourced with Outbox

The write path converts raw inputs — conversation turn pairs, documents, structured data — into conflict-resolved memories on disk and indexed nodes in the KG. It must survive process and machine crashes without silent data corruption. No distributed transaction exists across SQLite, Neo4j, and the filesystem; instead, Engram uses an event-sourced pipeline with outbox tables and idempotent steps.

> **Core invariant**
> SQLite is the only synchronous write target on the critical path. The ingest endpoint returns 200 OK as soon as the Event Ledger row is committed. Every downstream step is driven off the ledger by background workers. Each step is individually atomic at its target backend and idempotent on replay. Crash recovery is deterministic: the reconciliation worker drains stuck events and outboxes. Loss of Neo4j or filesystem content in isolation is recoverable because the filesystem is authoritative and Neo4j is a rebuild-from-filesystem derived index.

5.2 Ingest Unit — Message Pairs and Turn Groups

The ingest unit is a message pair — one user turn and its corresponding assistant turn — not an individual turn. Rationale:

- A single user turn in isolation often has no storable content ("ok, thanks").
- A single assistant turn in isolation may contain false statements; storing it as memory would poison future retrieval.
- The pair is the minimum unit with enough context to decide "is there a fact here, whose fact is it, and how confident should we be?"

When assistant turns contain tool calls, the unit extends to the full turn group: user → assistant(tool_call) → tool_result → assistant(final_response). The entire group is ingested atomically.

The ingest API accepts either a pair or a turn group. The pair_id is deterministic:

```
pair_id = sha256(session_id || user_turn_idx || assistant_turn_idx)
```

pair_id doubles as the idempotency key in the Event Ledger. Duplicate submissions are recognized and return 200 OK without re-processing.

5.3 Pipeline Overview

The pipeline has one synchronous step followed by a chain of asynchronous steps:

```
[Step 1 — sync, API thread]
  INSERT INTO events (event_id, pair_id, session_id, payload, status='RECEIVED')
    as a single SQLite transaction
  → return 200 OK with event_id
  [client done]

[Step 2 — async, Ingest Worker]
  Write-Path Gate (Core Model)
    → UPDATE events SET status='GATED_STORE' or 'GATED_SKIP'
    → if GATED_SKIP: pipeline terminates here

[Step 3 — async, Ingest Worker]
  S-R-O Extraction + L0 Abstract (Core Model, single call)
    → INSERT INTO extractions (event_id, resolved_text, triplets, l0_abstract)

[Step 4 — async, Ingest Worker]
  Entity Linking (Core Model + Gating Model embeddings)
    → resolve subject/object entities, produce node IDs
    → INSERT INTO linked_entities (event_id, subject_node_id, object_node_id, ...)

[Step 5 — async, Ingest Worker]
  Filesystem write (authoritative)
    → write to {data_dir}/{source_uri}/{semantic_filename}.md
    → atomic via temp-file + rename + fsync
    → INSERT INTO fs_outbox (event_id, source_uri, state='WRITTEN')

[Step 6 — async, Ingest Worker]
  Deduplication + Conflict Resolution + KG Index Update (Core Model + Neo4j)
    → inside one Neo4j transaction: MERGE node keyed on source_uri, create edges,
      resolve conflicts, update vector index
    → UPDATE fs_outbox SET state='INDEXED' WHERE event_id = ?

[Step 7 — async, Ingest Worker]
  Consolidation task enqueue
    → INSERT INTO consolidation_tasks (parent overview regen, manifest update)
    → UPDATE events SET status='COMPLETE'
```

5.4 Per-Step Specifications

5.4.1 Step 1: Event Recording

The API thread performs one SQLite INSERT wrapped in a transaction. The write is synchronous; on commit, the client receives 200 OK.

Attribute	Specification
Idempotency	UNIQUE constraint on pair_id. Duplicate submissions are caught at the constraint and the existing event_id is returned with 200 OK.

Attribute	Specification
Payload	JSON blob containing both turns of the pair (or turn group), session_id, timestamps, source (client/session-compact), and optional client-provided metadata.
Initial status	'RECEIVED'.
Latency budget	< 20 ms p99. SQLite WAL mode with one writer comfortably hits this.

5.4.2 Step 2: Write-Path Gate

The Core Model evaluates whether the turn pair contains storable information. This is a distinct prompt from the L0 read-path gate and operates on pair-level rather than query-level surface.

Attribute	Specification
Input	Full turn pair (user + assistant), preceding compacted session summary, session metadata
Output	{ "store": true | false, "reason": "..." }
On store=false	Update events.status = 'GATED_SKIP'. Pipeline terminates. The raw turn remains in the session cache for short-term context only.
On store=true	Update events.status = 'GATED_STORE'. Proceed to Step 3.
Failure handling	If Core Model returns malformed output, retry once with an error-correction prompt. On second failure, mark event FAILED; reconciliation worker will retry after a backoff.

5.4.3 Step 3: S-R-O Extraction + L0 Abstract

A single Core Model call produces coreference-resolved text, S-R-O triplets with confidence scores, and a one-sentence L0 abstract. This is the most information-dense Core Model call in the pipeline.

Attribute	Specification
Input	Resolved turn pair + preceding session context for coreference
Output	{ resolved_text, triplets[{subject, relation, object, confidence}], l0_abstract }
Confidence routing	triplet confidence ≥ 0.6 → standard KG edge. 0.3 ≤ confidence < 0.6 → create FACT node flagged LOW_CONFIDENCE. confidence < 0.3 → discard.

Attribute	Specification
Persisted to	extractions table in SQLite. This is the checkpoint boundary: if the pipeline crashes after Step 3, Step 4+ resume from the extractions row.

5.4.4 Step 4: Entity Linking

For each subject and object entity mentioned in the triplets, determine whether it is an existing KG entity or a new one.

Attribute	Specification
Candidate retrieval	Query Neo4j for ENTITY nodes matching on (a) exact name or canonical_name match from metadata.normalize.*, (b) vector similarity via Gating Model embedding of the entity name. Retrieve up to 5 candidates.
Thresholds	Cosine ≥ 0.92 AND name token overlap ≥ 0.5 → proceed to disambiguation. Below → treat as new entity. (Threshold raised from the earlier design's 0.88 to reduce false merges; see §5.7.)
Disambiguation	Always invoke the Core Model on any candidate above threshold, even if there is only one. The prompt includes the candidate's L0 abstract and the surrounding sentence from the current pair. The model returns matched_id or null.
New entity creation	If no match, create a new ENTITY node. Assign source_uri based on node type and entity name (see §6.1 for URI assignment).
Post-condition	linked_entities table holds the resolved node IDs. Triplets are keyed on those IDs before reaching Step 5.

5.4.5 Step 5: Filesystem Write

This is the authoritative write. The content becomes canonical the moment this step commits.

Attribute	Specification
Target path	{data_dir}/{source_uri_without_scheme}/{semantic_filename}.md where semantic_filename = {entity_or_episode_slug}_{one_line_summary_slug}.md
Atomicity	Write to {target}.tmp, fsync file, rename to {target}, fsync parent directory. POSIX rename is atomic within the same filesystem.

Attribute	Specification
File contents	YAML frontmatter (see §5.4.5.1) followed by resolved_text as Markdown body.
Outbox row	On successful rename, INSERT INTO fs_outbox (event_id, source_uri, state='WRITTEN', written_at). This row is the signal for Step 6 to proceed.
Idempotency	If the target file already exists with a matching content hash, treat as successful (duplicate pipeline execution). If the target exists with a different hash, log a conflict warning and proceed; Step 6's MERGE keyed on source_uri will handle the KG side correctly.

5.4.5.1 YAML frontmatter schema

```
---
id: 4b9f...-uuid
node_type: ENTITY          # ENTITY | EVENT | FACT | DOCUMENT | DIRECTORY
status: ACTIVE             # ACTIVE | HISTORICAL | LOW_CONFIDENCE
created_at: 2026-04-12T10:30:00Z
source_session_id: sess-abc-123
schema_version: 1
temporal:
  asserted_at: 2026-04-12T10:30:00Z
  valid_from: null
  valid_until: null
  phrase: "as of April 2026"
normalize:
  canonical_name: "Alice Chen"
  aliases: ["Alice", "A. Chen"]
provenance:
  extractor: core_model_v1
  confidence: 0.87
---
L0 abstract sentence goes here on the first body line.

Full resolved text follows as normal Markdown.
```

The frontmatter is read by both the KG-indexing step (to populate Neo4j properties) and the rebuild-from-filesystem command. Frontmatter is the canonical serialisation of every per-node field that is not content itself.

5.4.6 Step 6: Dedup, Conflict Resolution, KG Index Update

This is a single Neo4j transaction that combines dedup/conflict logic with the index update. It runs after the filesystem write has committed so that the KG always references files that exist on disk.

Attribute	Specification
Transaction scope	One Neo4j transaction: lookup existing edges for the subject-relation pair, classify as DUPLICATE | CONTRADICTION | CO_EXISTENCE, apply mutations, MERGE nodes keyed on source_uri, create CONTAINS/IS_PART_OF/RELATES_TO edges, update vector index.
Conflict classification	See §6.5 for the DUPLICATE / CONTRADICTION / CO_EXISTENCE decision rules and the SUPERSEDES edge semantics.
Idempotency	All MERGE operations are keyed on source_uri. Re-running the step produces the same result. Edge creation uses MERGE on (subject, relation_label, object) so duplicate edges are impossible.
On failure	UPDATE fs_outbox SET state='INDEX_FAILED', retry_count = retry_count + 1. Reconciliation worker retries with exponential backoff up to 3 attempts, then marks the event FAILED for manual inspection.
On success	UPDATE fs_outbox SET state='INDEXED', UPDATE events SET status='INDEXED'. Step 7 proceeds.

5.4.7 Step 7: Consolidation Enqueue

The final step enqueues background work. It is a bulk INSERT into consolidation_tasks and a status update on the event:

- CONSOLIDATE_OVERVIEW for the immediate parent directory, deduped against any PENDING task for the same directory.
- REGENERATE_MANIFEST for the same parent directory.
- PROPAGATE_OVERVIEW tasks for each ancestor up to the root, deduped.
- Optional: ATOMIZE / NORMALIZE / TEMPORALIZE tasks if the extraction flagged compound statements, unnormalized aliases, or missing temporal anchors.

The event transitions status='COMPLETE' on this step's commit.

5.5 Crash Recovery

Crashes are recovered deterministically by the Reconciliation Worker, which runs on a schedule (default: every 60 seconds) and at process startup.

Stuck state	Detection	Recovery action
events.status = 'RECEIVED' for > 5 min	Crash before Step 2 began, or Ingest Worker offline	Requeue for Ingest Worker. Idempotent.
events.status = 'GATED_STORE' with no extractions row	Crash during or after Step 3 failed to persist	Re-run Step 3. The pair payload is in the ledger.
fs_outbox.state = 'WRITTEN' for > 2 min	Crash between Step 5 and Step 6	Re-run Step 6. MERGE on source_uri is safe. If the file has since been superseded, the MERGE is a no-op.
fs_outbox.state = 'INDEX_FAILED' with retry_count < 3	Neo4j transient failure	Exponential backoff retry.
events.status = 'INDEXED' with no consolidation tasks for parent	Crash between Step 6 and Step 7	Re-run Step 7. Task inserts are deduped against PENDING.

Invariants enforced by the pipeline

- If a file exists on disk under source_uri, a KG node for that source_uri either exists or will exist once reconciliation drains.
- If a KG node exists, the file at its source_uri exists. (Step 5 commits before Step 6.)
- Every committed event reaches status='COMPLETE' or status='FAILED' eventually. No event is silently dropped.
- Re-running the pipeline on a completed event produces the same state (idempotency).

5.6 Failure Modes and Handling

Failure	Handling
Core Model malformed output (gate or extraction)	Retry once with error-correction prompt. On second failure, mark event FAILED and log. Manual replay via /api/v1/events/{id}/retry.
Core Model timeout (30 s per call, configurable)	Treat as malformed output. Retry once.
Filesystem I/O error at Step 5	Event remains in GATED_STORE. Reconciliation worker retries with exponential backoff. After 3 failures, mark FAILED.
Neo4j unavailable during Step 6	fs_outbox row stays at 'WRITTEN'. Reconciliation retries up to 3 times. The on-disk file is correct; only the index lags.
Neo4j unavailable during read path	Query endpoint returns 503. The L0 memory-hit fallback and L1 vector search both depend on Neo4j; there is no read path that bypasses it in Phase 1.
SQLite unavailable	Fatal for writes. API returns 503 on /api/v1/ingest. Reads may still succeed on cached sessions; new reads will fail on session lookup.

5.7 Entity Linking — Safer Defaults

Entity linking is the single step most prone to silent data corruption (merging two distinct people, or creating duplicates of the same person). Engram uses conservative defaults and explicit unmerge support.

- **Threshold.** Cosine ≥ 0.92 AND name token overlap ≥ 0.5 for any linking candidate. Raised from 0.88 because 384-dim BGE-Small embeddings of short names have high baseline similarity.
- **Disambiguation always runs.** Even for a single candidate above threshold, the Core Model is invoked with the candidate's L0 abstract and the surrounding sentence. Names alone do not decide.
- **Uncertainty defaults to new entity.** When the Core Model returns null (no match), a new entity is created. Incorrect separation is recoverable (a subsequent NORMALIZE task or explicit merge will combine them). Incorrect merging requires unmerge.
- **Unmerge operation.** A POST /api/v1/memories/{id}/unmerge endpoint splits a merged entity back into its contributing source facts, re-creating separate ENTITY nodes for each original extraction. See §8.6.

6. Knowledge Graph Schema

6.1 The mem:// Filesystem

Every stored memory has a canonical location on disk, identified by a mem:// URI. The URI is both the unique identifier of the memory node within the KG and the locator for the original content on the filesystem. The KG mirrors the filesystem hierarchy as a traversable graph: navigating the KG is equivalent to a depth-first traversal of the filesystem tree.

6.1.1 URI structure

```
mem://
├── user/                          # User-scoped memories
│   ├── identity/
│   │   └── profile.md
│   ├── preferences/
│   │   ├── coding_style.md
│   │   └── communication.md
│   ├── entities/
│   │   ├── alice/
│   │   │   ├── .manifest                  # one-line-per-child ToC
│   │   │   ├── overview.md                # directory-level overview (~2k tokens)
│   │   │   └── alice_ml-engineer-at-meta.md  # full document (semantic filename)
│   │   └── project_atlas/
│   │       ├── .manifest
│   │       ├── overview.md
│   │       └── project_atlas_distributed-ml-pipeline.md
│   └── episodes/
│       ├── .manifest
│       ├── overview.md
│       ├── 2026-03-10_moved-to-canada.md
│       └── 2026-03-15_started-new-role.md
├── agent/                         # Agent-scoped memories
│   ├── cases/
│   └── patterns/
└── shared/                        # Shared knowledge
    ├── facts/
    └── resources/
        ├── .manifest
        ├── overview.md
        ├── api_docs/
        │   ├── overview.md
        │   └── endpoints_rest-api-v2.md
        └── meeting_notes/
            ├── overview.md
            └── 2026-03-12_standup.md
```

6.1.2 Semantic filenames

Filenames encode a one-line semantic summary, allowing cheap listing without reading content:

```
Entity files:    {entity_slug}_{summary-slug}.md
Episode files:   {YYYY-MM-DD}_{summary-slug}.md
Resource files:  {resource_slug}_{summary-slug}.md
```

The Core Model performs a cheap ls-equivalent by listing a directory's children and reading their filenames and .manifest entries — no content load required.

6.1.3 .manifest files

Every directory contains a .manifest file: a newline-delimited list of child entries, each line formatted as:

```
alice_ml-engineer-at-meta.md | Alice is an ML engineer at Meta working on recommendation systems
project_atlas/ | Distributed ML pipeline project, started Q1 2026, Python + Ray
```

The Core Model reads a single .manifest to decide which children to descend into. This is cheaper than running N vector queries. Manifests are regenerated asynchronously whenever a child is added, removed, or superseded, with a 5-second debounce to coalesce rapid successive writes.

6.1.4 overview.md files

Every DIRECTORY node has an overview.md file (~2k tokens, bounded by overview_max_tokens in configuration) that summarizes its children, their relationships, and cross-references. Overviews serve three purposes:

- **L3 retrieval content.** At L3 in the cascade, the Core Model reads overview.md files to get richer context than L0 abstracts without the cost of full documents.
- **Hierarchical rollup.** mem://user/overview.md summarizes everything known about the user; mem://user/entities/overview.md summarizes all known entities. Broader context is cheap.
- **Consolidation mechanism.** Regenerating an overview forces the system to find connections between disparate child memories. This is the primary way Engram creates new cross-memory associations.

6.1.5 source_uri semantics

- **Unique identifier.** No two active nodes share a source_uri.
- **Stable.** Once assigned, a source_uri never changes. If a memory is superseded, the new memory gets a new source_uri; the old URI continues to point to the HISTORICAL version.
- **Locator.** Maps directly to a filesystem path under {data_dir}. The overview for mem://user/entities/alice/ is at {data_dir}/user/entities/alice/overview.md.
- **Hierarchy encoding.** Path segments encode containment and are queryable either by URI prefix matching or by graph traversal along CONTAINS edges.

6.2 Node Schema

All memory nodes share a fixed set of core properties. Domain-specific attributes live in the metadata MAP with a reserved-keys contract (§6.3).

Property	Type	Required	Description
id	UUID	Yes	Globally unique, immutable.
node_type	Enum	Yes	ENTITY | EVENT | FACT | DOCUMENT | DIRECTORY | SESSION_SUMMARY
status	Enum	Yes	ACTIVE | HISTORICAL | LOW_CONFIDENCE
source_uri	String	Yes	mem:// URI. Unique. Never changes.
parent_uri	String	No	mem:// URI of parent DIRECTORY. Null for root-level nodes.
l0_abstract	String	Yes	One-sentence summary. Indexed (full-text).
l0_embedding	Float[384]	Yes	BGE-Small mean-pool embedding. Vector-indexed.
retrieval_weight	Float	Yes	Decay-computed (0.05–1.0). Default 1.0 on creation.
overview_generated_at	Timestamp	No	DIRECTORY nodes only: last overview.md regen time.
created_at	Timestamp	Yes	Creation time.
last_accessed_at	Timestamp	Yes	Updated on every retrieval hit.
access_count	Integer	Yes	Retrieval hit count. Default 0.
source_session_id	UUID	No	Session that created this node.
superseded_by	UUID	No	For HISTORICAL nodes: the replacement node.
superseded_at	Timestamp	No	When supersession occurred.

Property	Type	Required	Description
schema_version	Integer	Yes	Schema version at creation. Phase 1: 1.
metadata	MAP	No	Reserved-keys map; see §6.3.

No l1_overview or l2_content as KG properties
Overview and full-document content are read from disk at {source_uri}/overview.md and the L2 filename path respectively. Neither is stored as a Neo4j property. Rationale: (1) Neo4j is designed for graph topology and small properties, not document bodies; (2) duplicating ~2 KB of overview and ~10 KB of full text per node across a million nodes blows up Neo4j's page cache with no query benefit (neither property is indexed); (3) single-authoritative-source semantics require exactly one place to hold the content, and that place is the filesystem.

6.3 Metadata Map — Reserved Keys

The metadata MAP is flexible but not free-form. Engram reserves a namespace of keys whose types and semantics are fixed. Reserved keys are validated at write time against the schema below; unreserved keys are permitted but not schema-validated.

Reserved key	Type	Semantics
schema_version	Integer	Always present. Used for migrations.
temporal.asserted_at	ISO8601	Timestamp at which this fact was asserted.
temporal.valid_from	ISO8601 | null	Start of the fact's validity window. Null if unbounded.
temporal.valid_until	ISO8601 | null	End of the fact's validity window. Null if ongoing.
temporal.phrase	String	Human-readable temporal anchor ("as of March 2026").
normalize.canonical_name	String	Canonical form produced by NORMALIZE task.
normalize.aliases	String[]	Known aliases / surface forms.

Reserved key	Type	Semantics
provenance.extractor	String	Extractor identifier: 'core_model_v1', 'frontier_bootstrap'.
provenance.confidence	Float	Original extraction confidence [0.0, 1.0].
provenance.ingest_event_id	UUID	Event Ledger row that produced this node.

Reserved-key validation is enforced by the ingest worker before Step 6. Writes that violate the schema are rejected; the event is marked FAILED with a schema_violation error.

6.4 Edge Schema

Edge type	Purpose	Example
CONTAINS	Parent directory contains child node	(mem://user/entities/) -[CONTAINS]→ (mem://user/entities/alice/)
IS_PART_OF	Inverse of CONTAINS	(alice) -[IS_PART_OF]→ (user/entities/)
RELATES_TO	Semantic relationship (extracted from content)	(Alice) -[RELATES_TO {label: 'works_at'}]→ (Meta)
SUPERSEDES	Conflict resolution: new version replaces old	(new_fact) -[SUPERSEDES]→ (old_fact)
REFERENCES	Cross-reference between nodes at any level	(meeting_notes) -[REFERENCES]→ (project_atlas)

6.4.1 Edge properties (common to all types)

Property	Type	Description
relation_label	String	Human-readable label from a controlled vocabulary (see §6.4.2). Structural edges use their type name.
confidence	Float	Extraction confidence [0.0, 1.0]. Always 1.0 for structural edges (CONTAINS, IS_PART_OF, SUPERSEDES).
status	Enum	ACTIVE | HISTORICAL.

Property	Type	Description
created_at	Timestamp	Creation time.
superseded_at	Timestamp	When superseded; null while ACTIVE.
superseded_by	UUID	Edge id that replaced this one; null while ACTIVE.
source_session_id	UUID	Session that created this edge; null for structural edges created on directory creation.

6.4.2 Relation label vocabulary

RELATES_TO edges use a controlled vocabulary to prevent fragmentation (works_at, worksAt, employed_by, works at, etc.). The vocabulary is a deployment-tunable list under prompts/relations.yaml. The NORMALIZE task (§7.2) maps extracted relation strings to the nearest canonical form by cosine similarity against pre-computed label embeddings. Extracted labels that fail to match any canonical form above threshold 0.85 are preserved verbatim and flagged for manual review via /api/v1/consolidation/status.

6.5 Conflict Resolution

When a new triplet is extracted, the dedup/conflict step (§5.4.6) classifies it against existing edges:

Case	Detection	Resolution
DUPLICATE	Same subject, equivalent relation (label OR cosine > 0.90 on relation-label embeddings), same/near-identical object (cosine > 0.95 on object l0_embedding)	Merge: increment access_count and update last_accessed_at on the existing edge. Update source_session_id if absent. Discard incoming triplet.
CONTRADICTION	Same subject, equivalent relation, different object (cosine ≤ 0.95)	Supersede: mark old edge status='HISTORICAL', superseded_by = new_edge.id, superseded_at = now. Mark old object node HISTORICAL if no remaining ACTIVE edges reference it. Create new edge + new object

Case	Detection	Resolution
		node with status='ACTIVE'. Create a SUPERSEDES edge from the new edge to the old.
CO_EXISTENCE	Same subject, different relation (or same relation but object cosine between 0.5 and 0.95 — plausible alternative, not a contradiction)	Write new edge with status='ACTIVE'. No modification to existing edges.

6.5.1 "Orphaned" definition

A node is orphaned when no ACTIVE edge in the graph references it as subject or object. Orphan detection runs as a single Cypher MATCH at supersession time. Orphaned nodes inherit status='HISTORICAL' — they remain queryable via explicit history lookups but are excluded from default retrieval filters.

6.6 Indexes

```
-- Vector index over L0 embeddings
CREATE VECTOR INDEX l0_idx IF NOT EXISTS
  FOR (n:Node) ON (n.l0_embedding)
  OPTIONS {
    indexConfig: {
      `vector.dimensions`: 384,
      `vector.similarity_function`: 'cosine'
    }
  };

-- BM25 full-text index over L0 abstracts
CREATE FULLTEXT INDEX l0_text_idx IF NOT EXISTS
  FOR (n:Node) ON EACH [n.l0_abstract];

-- URI prefix index for filesystem-style navigation
CREATE TEXT INDEX uri_prefix_idx IF NOT EXISTS
  FOR (n:Node) ON (n.source_uri);

-- Status index (used for default ACTIVE-only filters)
CREATE INDEX status_idx IF NOT EXISTS
  FOR (n:Node) ON (n.status);

-- Retrieval weight index (for decay-filtered queries)
CREATE INDEX weight_idx IF NOT EXISTS
  FOR (n:Node) ON (n.retrieval_weight);
```

7. Memory Consolidation

7.1 Overview

Consolidation is the set of background processes that (1) normalize and enrich newly ingested facts, (2) maintain hierarchical summaries (overview.md, .manifest), and (3) create new connections between disparate memories. All consolidation runs as background tasks from the SQLite consolidation_tasks queue.

7.2 Task Queue

```
CREATE TABLE consolidation_tasks (
  task_id       TEXT PRIMARY KEY,
  node_id       TEXT NOT NULL,
  task_type     TEXT NOT NULL,       -- see §7.3
  status        TEXT NOT NULL DEFAULT 'PENDING',  -- PENDING | PROCESSING | COMPLETE |
FAILED
  priority      INTEGER NOT NULL DEFAULT 5,       -- 1 (highest) to 10 (lowest)
  scheduled_at  TEXT NOT NULL,
  started_at    TEXT,
  completed_at  TEXT,
  retry_count   INTEGER NOT NULL DEFAULT 0,
  error_message TEXT,
  created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

-- Dedup constraint: at most one PENDING or PROCESSING task per (node_id, task_type)
CREATE UNIQUE INDEX idx_tasks_pending_unique
  ON consolidation_tasks(node_id, task_type)
  WHERE status IN ('PENDING', 'PROCESSING');

CREATE INDEX idx_tasks_status ON consolidation_tasks(status, priority, scheduled_at);
```

7.3 Task Types

Task type	Description
ATOMIZE	Split compound statements into atomic triplets. "Alice moved to Canada and started at Meta" becomes two triplets with independent lifecycles.
NORMALIZE	Standardize entity names and relation labels. Merges aliases ((Robert, Bob), (NYC, New York City)) and maps relation strings to controlled-vocabulary labels. Updates metadata.normalize.canonical_name and metadata.normalize.aliases; rewrites RELATES_TO edges where needed.
TEMPORALIZE	Enrich L0 abstract with a human-readable time phrase and set metadata.temporal.* from session metadata and content signals.

Task type	Description
INTEGRATE	Write atomized, normalized, temporalized facts back to the KG and filesystem. Creates/updates nodes, edges, and the directory's .manifest.
CONSOLIDATE_OVERVIEW	Regenerate overview.md for a directory. Reads children's L0 abstracts and relations, identifies cross-connections, writes a new overview. Updates overview_generated_at on the DIRECTORY node.
REGENERATE_MANIFEST	Rebuild the .manifest file for a directory from current child listings.
PROPAGATE_OVERVIEW	After a CONSOLIDATE_OVERVIEW completes, enqueue the same for the parent directory, recursively up to the root. Each level is deduped via the unique PENDING/PROCESSING index.

7.4 Triggers

- **On write (Step 7).** CONSOLIDATE_OVERVIEW + REGENERATE_MANIFEST for the immediate parent. PROPAGATE_OVERVIEW for each ancestor up to root.
- **On session COMMITTING.** ATOMIZE / NORMALIZE / TEMPORALIZE tasks for each significant turn pair in the session history.
- **On session compaction (/compact).** When session context hits window_threshold_ratio, the Session Manager compacts the session for continued use (Core Model call; see §8.3) and schedules ingest pipeline runs for every turn pair leaving the window. The consolidation queue receives the downstream tasks automatically via Step 7.
- **Daily reconciliation cron.** A background job scans DIRECTORY nodes where any child's created_at or superseded_at is newer than the directory's overview_generated_at. Any found are enqueued for CONSOLIDATE_OVERVIEW (deduped).

7.5 Backpressure and Debouncing

Naïve propagation ("write enqueues one overview per ancestor") at ingest rate R generates ~R · tree_depth consolidation tasks per second. For a deep tree and a bursty ingest source this outpaces the consolidation worker. Engram applies two measures:

- **Debounce.** After a directory receives its first CONSOLIDATE_OVERVIEW enqueue, subsequent enqueues within overview_debounce_seconds (default 30) are dropped

(guaranteed by the unique-PENDING index). Writes during the debounce window are implicitly covered by the pending regeneration.

- **Backpressure.** The consolidation_tasks queue depth is exposed as a metric. If depth exceeds max_backlog (default 10,000), the ingest endpoint returns 503 Service Unavailable with a Retry-After header until depth falls below max_backlog / 2. This prevents unbounded memory growth and lets the consolidation worker catch up.

Later phases will replace the SQLite queue with Redis streams or a Postgres LISTEN/NOTIFY pipeline; the contract between ingest and consolidation is queue-agnostic.

7.6 Breadth-First Tree Rendering

When the Core Model needs to view the mem:// tree as part of its L1 input, the system renders the tree under a token budget (default tree_display_max_tokens = 2048). Rendering is breadth-first, not depth-first.

> **Why breadth-first**
> A depth-first render exhausts its token budget drilling into one branch and leaves the Core Model unaware of siblings it could alternatively explore. The Core Model is best served by knowing the shape of the whole tree at coarse granularity, then issuing explicit ls or overview commands on the branches it wants to investigate.

7.6.1 Rendering algorithm

18. Render the root and its immediate children at level 1 with each child's L0 abstract truncated to max(80, per_node_budget) tokens.

For levels L = 2 .. max_display_depth: compute per_node_budget = remaining_budget / expected_nodes_at_level_L. If per_node_budget < MIN_NODE_TOKENS (default 20), stop descending and emit "... (N more at depth L, use `ls` or `overview` to explore)" markers.

Always render at least depth 2 regardless of budget; if budget is insufficient, include the abstracts truncated to the bare filename summary.

Always include ellipsis markers for truncated branches so the Core Model knows what it is missing.

After the render, the Core Model typically responds with CLI-style commands (ls, overview, cat — see §8.4) to drill into specific branches as part of its retrieval plan.

8. Core Model

8.1 Interface

The Core Model is accessed through an abstract interface (CoreModelProvider) that exposes a single method: complete(system_prompt, user_prompt, output_schema) → structured JSON. Concrete implementations wrap local inference (Qwen3.5-0.8B via llama.cpp, vLLM, or TGI) or remote API providers (Anthropic, OpenAI, Ollama) behind the same contract.

The provider is responsible for:

- Applying grammar-guided decoding or a JSON-schema constrained output mode to produce valid structured output.
- Retrying once on malformed output with an error-correction prompt.
- Enforcing the configured timeout (default 30 s per call).
- Emitting structured logs (task_type, prompt hash, tokens in/out, latency, success/failure) for telemetry and for the continuous-improvement loop (§14.3).

8.2 Task Inventory

All Core Model tasks are listed below with their prompt file, input, and output schema. Prompts are Jinja2 templates under prompts/; each is versioned, and the version is included in the prompt preamble so logs can correlate outputs to prompt revisions.

Task	Prompt file	Input variables	Output schema
Write-path gate	prompts/gate_write.j2	turn_pair, session_summary	{ store: bool, reason: str }
S-R-O + L0 abstract	prompts/extract.j2	turn_pair, session_context	{ resolved_text, triplets[], l0_abstract }
L1 plan	prompts/l1_plan.j2	query, session_context, tree_render, kg_schema, memory_hit?	See §3.2.1
L2/L3/L4 plan-with-judge	prompts/ln_plan.j2 (single template, depth parameterized)	query, session_context, previous_results, kg_schema	See §4.2
Coverage assessment (fused)	same prompt — coverage is embedded in plan output	—	—

Task	Prompt file	Input variables	Output schema
Dedup + conflict	prompts/dedup.j2	incoming_triplet, existing_edges[]	{ case: DUPLICATE | CONTRADICTION | CO_EXISTENCE, existing_edge_id?, reason }
Entity linking	prompts/entity_link.j2	entity_name, surrounding_sentence, candidates[]	{ matched_id | null, confidence, reason }
Overview generation	prompts/overview.j2	directory_uri, children_abstracts[], children_relations[]	Markdown overview string (≤ overview_max_tokens)
Session compaction	prompts/session_compact.j2	turn_history[], compaction_budget	{ compacted: str, key_facts[], key_entities[] }
Unmerge (manual)	prompts/unmerge.j2	merged_node, source_extractions[]	{ splits: [{name, l0_abstract, triplets}] }

No reasoning field in L1/Ln plan outputs
The production plan schema does not include a reasoning or chain_of_thought field. Reasoning is only elicited during the bootstrap data-collection phase (§14.2) to aid manual trace filtering; it is stripped from the SFT target so the deployed Qwen3.5-0.8B never emits it. This saves ~300–500 ms per plan call at the 128-token reasoning length typical of such models.

8.3 Session Compaction (formerly "session summarize")

The naming distinction matters: the operation restructures and shortens session history, preserving key facts and entities. It is not merely a lossy summary. The prompt is designed to produce a compacted block that a subsequent ingest pipeline run can safely consume.

8.3.1 Compaction trigger

Triggered when (a) session context exceeds window_threshold_ratio of the frontier context window (default 0.4), or (b) the user explicitly sends /compact.

8.3.2 Compaction flow

18. Session Manager selects turn pairs 1..k to compact (the oldest turns leaving the window).
19. Core Model is called with prompts/session_compact.j2 on those turn pairs. Output: { compacted: str, key_facts, key_entities }.
20. The compacted string replaces turns 1..k in the session cache (Redis).
21. Concurrently, the Session Manager enqueues each of the original turn pairs (uncompacted) for ingest via /api/v1/ingest. These inherit source='session_compact' in their Event Ledger payload.
22. Downstream, those ingest events run the full pipeline (write-path gate → extraction → filesystem → KG index → consolidation).

Compaction ingests from the uncompacted turns, not from the compact
It would be tempting to ingest from the compact itself (cheaper, fewer LLM calls). This is wrong: the compact is a lossy, reorganized view optimized for short-term context carry-over, and extracting triplets from it loses detail and temporal precision. The authoritative ingest must operate on the original turn pairs.

8.4 CLI-Style Traversal Commands

The Core Model's retrieval plans are expressed as sequences of CLI-style commands. This is the canonical action vocabulary: L1, L2, L3, and L4 prompts all output plans of the same command type; the level affects which commands are valid and how the orchestrator schedules them.

Command	Description
tree <uri>	Renders the hierarchical structure under a URI breadth-first, respecting the token budget (<rich-text font-family="Century Schoolbook" italic="true">tree_display_max_tokens</rich-text>). Useful for high-level structure checks.
ls <uri>	List children of a directory node via CONTAINS edges. Returns source_uri + L0 abstract per child. Equivalent to reading the .manifest.
cat <uri>	Read the full L2 content of a leaf node from the filesystem.
overview <uri>	Read the overview.md for a directory node.
find <query> [--scope <uri>] [--k N]	Vector search over l0_embedding, optionally scoped to a subtree by URI prefix. k defaults to 10, capped by max_l1_vector_results.

Command	Description
rel <node_id> [--type <edge_type>] [--hops N]	Follow semantic edges from a node. N defaults to 1, capped at 4.
history <node_id>	Follow SUPERSEDES chain to get full version history. Returns HISTORICAL nodes by design.
template <name> [params...]	Invoke a parameterized Cypher template (see §8.5). This is the only way to run Cypher in Phase 1.

Raw Cypher is not available to the Core Model. There is no cypher <query> command. The template system (§8.5) is the only Cypher surface.

8.5 Cypher Template Library

Every non-trivial Cypher query executed on behalf of the Core Model is a parameterized template from a deployer-reviewed library. This converts an open-ended code-generation problem (generate safe Cypher) into a constrained selection problem (pick a template, fill parameters). The template library is the security boundary (§12.3) and the training surface for the Core Model's Cypher output.

8.5.1 Initial template set

Template	Purpose
t_children_of (uri)	ls equivalent: list ACTIVE children by CONTAINS from a directory URI.
t_overview_for (uri)	Fetch the overview content (reads filesystem, not Cypher, but exposed as a template for uniformity).
t_neighbours_by_relation (node_id, relation, hops=1)	Follow RELATES_TO edges of a given label up to N hops. N capped at 4.
t_path_between (src_id, dst_id, max_hops=4)	Shortest path between two nodes across structural and semantic edges.

Template	Purpose
t_temporal_filter (node_id, from?, until?)	Filter RELATES_TO edges of a node by metadata.temporal.valid_from / valid_until.
t_history_chain (node_id)	Follow SUPERSEDES to get the supersession chain. Returns HISTORICAL nodes on purpose.
t_find_by_uri_prefix (prefix, limit=20)	URI-prefix scoped list; the "cd and ls" equivalent for a subtree.
t_cross_references (node_id, limit=10)	REFERENCES edges from a node.
t_top_k_vector (query_embedding, k=10, scope_prefix?)	Vector search with optional URI prefix scope.

8.5.2 Template guarantees

- **Read-only.** All templates are executed under the Neo4j Engram_reader role (§12.3), which lacks write privileges. Any template that attempts a write fails at the database level regardless of what the Core Model emits.
- **Bounded.** Every template includes a LIMIT clause (caller-provided or defaulted) and a hops cap.
- **Status filter by default.** Every template includes WHERE n.status = 'ACTIVE' unless it is explicitly a history-scoped template (t_history_chain).
- **Query timeout.** All template calls are wrapped with a Neo4j driver-level timeout (5 s at L2, 15 s at L4, configurable). On timeout, the orchestrator proceeds with whatever it has.

8.6 Unmerge Operation

Entity merges are occasionally wrong. Unmerge is the recovery path. It is implemented as a dedicated Core Model task and a dedicated API endpoint.

23. Client calls POST /api/v1/memories/{id}/unmerge.
24. Orchestrator fetches all Event Ledger rows whose linked_entities row points to this merged node.
25. For each distinct source extraction, the Core Model is asked to regenerate a standalone entity record (prompts/unmerge.j2).
26. A new ENTITY node is created per unmerged source. The original merged node is marked HISTORICAL with superseded_by pointing to all splits (multi-valued

supersession). Edges from the original are re-pointed to the appropriate split based on the originating extraction.

27. Consolidation tasks are scheduled on the affected parent directory to regenerate overview and manifest.

Unmerge is non-destructive: the merged node's HISTORICAL version remains queryable via t_history_chain.

9. Session Management

9.1 Session Lifecycle

State	Description	Transitions
ACTIVE	Receiving messages. Full session history in cache.	→ WINDOWED on threshold, → COMMITTING on close/timeout
WINDOWED	History exceeds window_threshold_ratio. Older turn pairs compacted (see §8.3); recent turns in raw form. Consolidation pipeline runs on compacted turns.	→ COMMITTING on close/timeout
COMMITTING	Session end triggered. Remaining raw turn pairs flushed through ingest pipeline. A SESSION_SUMMARY node is written.	→ COMMITTED
COMMITTED	All LTM extractions complete. Session no longer in cache.	Terminal

9.2 Session Cache

The session cache is Redis in Phase 1. The in-memory dictionary implementation is retained for single-process development only and is explicitly refused at startup when server.workers > 1.

Attribute	Specification
Backend (default)	Redis 7+ at session_cache.redis_url.

Attribute	Specification
Key	session:{session_id}
Value	Serialized session state: { turns: [{role, content, timestamp, turn_idx}], status, created_at, compacted_turns_idx_upper_bound }
Timeout	30 minutes of inactivity (configurable session.timeout_minutes). Redis TTL is refreshed on every session write.
On timeout	Session Manager marks status=COMMITTING, drains remaining uningested turn pairs into ingest queue, writes SESSION_SUMMARY node, deletes the Redis key.
Window threshold	session.window_threshold_ratio (default 0.4) of frontier context window, OR session.max_turns_before_window (default 50). First to trigger wins.

9.3 Session Context in Retrieval

L1 planning (§3.2) receives the session context as an input. The full current state of the session cache (both compacted and raw portions) is passed. The Core Model may mark session_sufficient = true and short-circuit the cascade if the session context alone answers the query.

10. Soft Memory Decay

10.1 Formula

$$\text{retrieval\_weight} = \alpha \cdot \text{recency} + \beta \cdot \text{frequency} + \gamma \cdot \text{centrality}$$

where:

$$\text{recency} = \exp(-\lambda \cdot \text{days\_since\_last\_access})$$

$$\text{frequency} = \log(1 + \text{access\_count}) / \log(1 + \text{p95\_access\_count})$$

$$\text{centrality} = \text{relates\_to\_degree} / \text{p95\_relates\_to\_degree}$$

Two adjustments from the naive formula are worth calling out:

- **Percentile normalisation for frequency and centrality.** The original design used max_access_count and max_degree_in_graph as denominators. Both are dominated by a small number of outliers (super-hub structural nodes, extremely-revisited root overviews), making the ratio uselessly small for the median node. Using the 95th percentile gives a stable, meaningful normalisation.

- **Structural edges excluded from centrality.** Only RELATES_TO edges count. CONTAINS/IS_PART_OF are a function of the tree shape, not semantic importance, and including them would swamp the signal.

10.2 Presets

Deployment	α	β	γ	λ (half-life)	Rationale
Personal conversation	0.40	0.30	0.30	0.01 (~69 days)	Slow decay. Personal context stays accessible.
Coding agent	0.40	0.50	0.10	0.05 (~14 days)	Fast decay. Active code context dominates.
Knowledge base	0.20	0.30	0.50	0.005 (~138 days)	Very slow decay. Structural knowledge persists.

10.3 Execution

Decay recomputation runs as a daily cron job (decay.schedule, default 03:00 local time). The job:

Computes p95_access_count and p95_relates_to_degree over ACTIVE nodes. Cached for the duration of the run.

For each ACTIVE node, computes the new retrieval_weight and writes it back. Runs in batches of 10,000 nodes inside short Neo4j transactions to avoid long-running lock holds.

28. HISTORICAL nodes are exempt. Their retrieval_weight is fixed at 0.0.
29. Nodes below dormant_floor (default 0.05) are excluded from L1 first-pass vector retrieval but remain accessible at L2+ via explicit Cypher templates (which can opt into weight-inclusive lookups).

10.4 Dormant-Floor Filtering in Vector Search

Neo4j's native vector index does not support property-level filtering natively. Engram implements dormant-floor filtering as post-retrieval filtering in application code: top-K is over-fetched ($K \cdot 1.5$) from the vector index and filtered by retrieval_weight ≥ dormant_floor before being returned to the orchestrator. The 50% over-fetch keeps effective K stable under typical decay distributions.

11. REST API

11.1 Endpoints

Endpoint	Method	Description
/api/v1/query	POST	Submit a query. Returns MSC-assembled answer from Frontier LLM.
/api/v1/ingest	POST	Submit a turn pair or turn group for memory extraction. Returns event_id.
/api/v1/ingest/batch	POST	Submit up to 100 items. Returns job_id for polling.
/api/v1/sessions	POST	Create a new session. Returns session_id.
/api/v1/sessions/{id}	GET	Get session state (turn count, status, metadata).
/api/v1/sessions/{id}	DELETE	End session. Triggers COMMITTING → COMMITTED flow.
/api/v1/sessions/{id}/message	POST	Add a turn pair to an active session (combined ingest + context update).
/api/v1/sessions/{id}/compact	POST	Force a session compaction before the threshold.
/api/v1/memories	GET	List memory nodes. Cursor pagination.
/api/v1/memories/{id}	GET	Retrieve a specific node with frontmatter, overview link, and edges.
/api/v1/memories/{id}/retire	POST	Soft-delete: mark HISTORICAL with a tombstone. No hard delete endpoint in Phase 1.
/api/v1/memories/{id}/unmerge	POST	Split a merged entity back into its contributing sources (§8.6).
/api/v1/memories/{id}/history	GET	Full supersession chain.
/api/v1/events/{id}/retry	POST	Manually retry a FAILED ingest event.
/api/v1/consolidation/status	GET	Queue depth by task_type, counts of PENDING / PROCESSING / COMPLETE / FAILED.
/api/v1/consolidation/trigger	POST	Manually enqueue consolidation for a node or subtree.

Endpoint	Method	Description
/api/v1/health	GET	Readiness probe: checks Neo4j, Redis, Core Model, Gating Model.
/api/v1/config	GET	Current system configuration (non-sensitive fields).

11.2 Query Endpoint

11.2.1 Request

```
POST /api/v1/query
Authorization: Bearer {api_key}
Content-Type: application/json

{
  "session_id": "sess-abc-123",       // optional; null for stateless queries
  "query": "What project is Alice working on?",
  "max_depth": "L4",                  // optional override; default L4
  "max_reentries": 2                  // optional override; default from config
}
```

11.2.2 Response

```
200 OK
{
  "answer": "Alice is working on Project Atlas, a distributed ML pipeline...",
  "session_id": "sess-abc-123",
  "retrieval_metadata": {
    "cascade_depth_reached": "L2",
    "levels_visited": ["L1", "L2"],
    "predicted_depth": "L2",
    "nodes_retrieved": 3,
    "total_context_tokens": 1842,
    "reentries": 0,
    "coverage_per_level": {
      "L1": { "covered": 2, "missing": 1 },
      "L2": { "covered": 3, "missing": 0 }
    },
    "latency_ms": {
      "l0_gate": 18,
      "l1_plan": 1120,
      "vector_search": 43,
      "l2_plan_judge": 980,
      "msc_assembly": 15,
      "frontier_answer": 2430,
      "total": 4606
    }
  }
}
```

11.3 Ingest Endpoint

```
POST /api/v1/ingest
Authorization: Bearer {api_key}
Content-Type: application/json

{
  "session_id": "sess-abc-123",
  "turn_pair": {
    "user": { "content": "I just started a new job at Meta.", "timestamp": "2026-04-12T10:28:00Z", "turn_idx": 14
},
    "assistant": { "content": "Congratulations! What team are you on?", "timestamp": "2026-04-12T10:28:05Z",
"turn_idx": 15 }
  }
}

→

202 Accepted
{
  "event_id": "evt-0194-...",
  "pair_id": "d3a1f...-sha256",
  "status": "RECEIVED"
}
```

11.4 Authentication — Phase 1

Phase 1 ships with single-key bearer-token authentication. Every request must include Authorization: Bearer {api_key} where api_key is configured at deploy time. Requests without a valid key receive 401. This is the bare minimum that lets Engram be exposed on a port without becoming a resource-exhaustion target. Full multi-tenant auth, per-tenant rate limiting, and tenant-scoped source_uris are the subject of §12.

11.5 Rate Limiting

Phase 1 implements per-process token-bucket rate limiting on /api/v1/query (default 60 req/min) and /api/v1/ingest (default 600 req/min). Limits are enforced in the API layer using starlette middleware. When deployed behind a load balancer with multiple workers, consider upgrading to a Redis-backed distributed token bucket; the bucket implementation is pluggable.

12. Security

12.1 Authentication

Phase 1 authentication is a single bearer API key configured at deploy time (api.api_key in configuration, or Engram_API_KEY environment variable). Every endpoint except /api/v1/health requires the key. 401 is returned for missing or invalid keys.

Secrets handling:

- The API key, Neo4j password, and Frontier LLM API key are sourced from environment variables, not committed configuration files.
- Configuration files reference these via ${Engram_API_KEY} interpolation; the loader refuses to start if any required secret is unset.
- Later phases will add pluggable secrets backends (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets).

12.2 Tenancy Model

Phase 1 is single-tenant per deployment. The mem:// URI scheme retains its top-level partitions (user/, agent/, shared/) for semantic clarity, but all partitions belong to one logical tenant. Multi-tenancy is a Phase 2 concern and will introduce:

- A tenant_id prefix on every source_uri (mem://t_<tenant_id>/user/...).
- Tenant-scoped auth principals and per-tenant rate limits.
- Enforcement at the API layer: no request may access a source_uri outside its principal's tenant scope.

12.3 Neo4j Access Roles

The write pipeline and the read pipeline use separate Neo4j user roles to constrain blast radius in case of Core Model prompt injection or misbehaviour.

Role	Privileges	Used by
Engram_writer	Full read + write on the Node label. Used for ingest Step 6 and consolidation INTEGRATE tasks.	Ingest Worker, Consolidation Worker
Engram_reader	Read-only on the Node label. Cannot execute CREATE, MERGE, SET, DELETE, REMOVE, or CALL apoc.* procedures.	Retrieval Orchestrator — every Core-Model-driven Cypher template executes under this role.

Why this matters
Even though Core Model output is constrained to parameterised Cypher templates (§8.5), the database role is the final line of defence. A Core Model that somehow

produced write Cypher (via a template bug, prompt injection, or a compromised template library) still cannot mutate the graph under Engram_reader. Run the reader role as restrictive as possible.

13. Operations & Telemetry

13.1 Availability in Phase 1

Phase 1 is single-node per storage backend: one Neo4j, one Redis, one SQLite file, one filesystem. This is acceptable for a personal-assistant or single-team deployment and not acceptable for public multi-user production. Operators must plan accordingly:

- **RPO.** Up to the filesystem snapshot interval (see §13.4). Committed memories are durable on disk.
- **RTO.** Minutes for Neo4j restart; hours for full disaster recovery including KG rebuild from filesystem.
- **Scaling path.** Phase 2 introduces Neo4j Causal Cluster or a graph alternative with built-in HA, Redis Sentinel or Cluster, and a replicated filesystem (NFS+rsync, object storage, or distributed FS).

13.2 Telemetry Requirements

The following metrics must be emitted per request and aggregated for monitoring. All counters and histograms are Prometheus-compatible by default.

Metric	Purpose
Engram_query_latency_seconds {level, phase}	Histogram of per-phase latencies (L0 gate, L1 plan, vector search, Ln plan-judge, MSC assembly, frontier answer).
Engram_query_depth_predicted_vs_reached	Two-dimensional counter: (predicted, actual_terminal_depth). The basis for the shallow-bias hit-rate alarm.
Engram_reentries_per_query	Histogram.
Engram_l0_gate_decisions{bypass_type}	Counter of BYPASS / CONTINUE / OVERRIDDEN_BY_MEMORY_HIT / OVERRIDDEN_BY_REGEX.

Metric	Purpose
Engram_ingest_pipeline_stage_seconds{stage}	Per-step duration in the ingest pipeline.
Engram_ingest_events_total{final_status}	Counter of RECEIVED → COMPLETE / GATED_SKIP / FAILED.
Engram_consolidation_queue_depth	Gauge, sampled every 10 s.
Engram_consolidation_task_duration_seconds{task_type}	Histogram per task type.
Engram_core_model_calls_total{task, provider}	Calls and token counts per Core Model task.
Engram_frontier_tokens_total{direction}	Frontier LLM tokens in / out.
Engram_kg_node_count, Engram_kg_edge_count	Gauges for size monitoring.

13.5 Schema Migrations

Every node and edge carries schema_version. Migrations are authored as numbered scripts under migrations/ that (a) update frontmatter on disk, (b) rebuild the affected Neo4j properties. Running a migration:

30. Put ingest into maintenance mode (returns 503; existing queries served from current state).
31. Drain the ingest and consolidation workers.
32. Run the migration script against the filesystem. Increments schema_version on touched nodes.
33. Run Engram rebuild-kg --incremental --since {migration_start} to update Neo4j.
34. Resume workers.

Because the filesystem is authoritative, rollback is a restore of the pre-migration snapshot.

14. Model Training

14.1 Gating Classifier

14.1.1 Objective

Train a 33M-parameter binary classifier that labels queries as Class 0 (self-contained, no retrieval) or Class 1 (needs retrieval). The classifier is separate from the embedding model (dual configuration — §1.5). The unified configuration's training procedure is provided in §14.1.5 for completeness.

14.1.2 Training data

Source	Size	Label strategy	Notes
CANARD dataset	~40K	Rewritten questions → Class 0; original context-dependent → Class 1	Primary public source.
Synthetic standalone factuals	~10K	Class 0	Diverse factual questions requiring no memory.
Synthetic conversational follow-ups	~10K	Class 1	Anaphora, deixis, ellipsis.
Hard negatives (looks-dependent, is-standalone)	~5K	Class 0	Trick the classifier into recall.
Personal-memory probes (synthesized)	~5K	Class 1	"what's my wife's birthday" pattern — linguistically self-contained but memory-dependent.

Total: ~70K labelled examples, 80/10/10 split. Personal-memory probes address the C6-class failure mode from the read-path design; see §14.1.4 for the complementary edge-case model.

14.1.3 Training procedure

- **Architecture.** Frozen BGE-Small backbone for embedding use elsewhere in the system. A separate 33M-parameter classifier (also BERT-based, unshared weights) with a linear head (nn.Linear(384, 2)) is trained from scratch or initialized from BGE-Small weights.
- **Loss.** Binary cross-entropy with class weights; Class 1 weighted at 1.5× to bias toward recall.

- **Hyperparameters.** LR 2e-5, linear warmup 10% of steps, cosine decay. Batch 64. Epochs 5. Max sequence 128 tokens. Weight decay 0.01.
- **Evaluation.** Primary: Recall on Class 1 at threshold 0.3 (target ≥ 0.97). Secondary: F1 at threshold 0.5 (target ≥ 0.92).

14.1.4 Edge-case model (optional, later-phase)

Operators who observe sustained false-negatives from the main classifier on personal-memory queries can train a complementary specialized model:

- Scope: binary classifier trained only on "looks-self-contained-but-requires-memory" patterns (e.g. possessive probes, personal fact lookups).
- Architecture: same 33M encoder, separate head.
- At inference: if the main classifier returns Class 0, the edge-case model is consulted; a positive verdict overrides to Class 1.
- Training data is deployment-specific and must be generated synthetically from the target agent's expected usage patterns.

In practice the memory-hit vector fallback (§3.1.2) catches most of the same failures at a fraction of the complexity. The edge-case model is worth deploying only when residual errors persist after the fallback is in place.

14.2 Core Model Training

14.2.1 Overview

Core Model training follows three phases: (A) bootstrap with a Frontier LLM acting as Core Model to collect traces, (B) supervised fine-tuning (SFT) on the traces, (C) Direct Preference Optimisation (DPO) on preference pairs derived from retrieval outcomes.

14.2.2 Phase A — Bootstrap

Configure Engram with core_model.provider set to an API frontier model. The system operates normally, but every Core Model call is served by the frontier. Log every call with task_type, system_prompt, user_prompt, structured_output, chain_of_thought_rationale (bootstrap-only field — stripped before SFT), latency, and the downstream success signal (did the Frontier LLM answer ANSWER or NEED_MORE; did the ingest pipeline complete).

Target trace distribution:

Task	Target traces	Notes
Write-path gate	8K–10K	Balanced true/false. Include tricky cases.
S-R-O + L0	15K–20K	Diverse entity types, relation types, coreference patterns.
L1 plan	10K–15K	Balanced across all depth levels. Include all three modes (AGFS, KG, HYBRID).
Ln plan-with-judge	10K–15K	Each of L2, L3, L4 plans with fused judgment. Balanced sufficient/insufficient outcomes.
Dedup + conflict	5K–8K	Balanced DUPLICATE / CONTRADICTION / CO_EXISTENCE.
Entity linking	5K–8K	Near-duplicates, abbreviations, aliases.
Overview generation	3K–5K	Diverse document types and directory sizes.
Session compaction	2K–3K	Varying session lengths.
Unmerge	0.5K–1K	Rare in practice; augment with synthetic splits.

14.2.3 Synthetic augmentation for conflict resolution

Natural conversation data under-represents contradictions and near-duplicates. Synthesize:

- Contradiction pairs (~2,000): entity attribute changes ("I work at Google" → "I just started at Meta"). Diverse relation types.
- Near-duplicate entities (~1,000): (Robert, Bob), (NYC, New York City), (Project Atlas, the atlas project).
- Temporal update sequences (~500): a fact updates 3–4 times, forming a supersession chain.
- Co-existence examples (~1,000): multiple independent true facts about the same entity.

14.2.4 Phase B — Supervised Fine-Tuning

- **Base model.** Qwen/Qwen3.5-0.8B (default) or Qwen2.5-0.5B-Instruct (lower-resource deployments).
- **Target format.** (system_prompt + user_prompt, output). Task-type tags prepended to system_prompt ([GATE], [EXTRACT], [L1_PLAN], [LN_PLAN], [DEDUP], [LINK], [OVERVIEW], [COMPACT]). Reasoning field from bootstrap is stripped — the target is the structured output schema only.

- **Training.** LoRA rank 64, alpha 128, target all linear layers. LR 1e-4 cosine. Batch 8 (grad accumulation to 64). 3 epochs. Max seq 2048.
- **Grammar-guided decoding.** At inference, structured outputs are constrained to valid JSON schema via Outlines or lm-format-enforcer. Cypher template selections use a small enum over template names, which grammar-guidance handles trivially.
- **Evaluation.** Per-task schema validity > 98%. Semantic correctness (cosine vs gold) > 0.90. Template-selection correctness (multi-class F1) > 0.90.

14.2.5 Phase C — DPO

Run the SFT model on ~5,000 held-out queries at temperature 0.7, generating 4 completions per query. Score each by the downstream outcome: did the resulting MSC enable the Frontier LLM to produce ANSWER on the first pass? Form (winner, loser) pairs across completions of the same query. DPO $\beta = 0.1$, LR 5e-6, 1 epoch.

Acceptance criteria after DPO:

- End-to-end answer correctness ≥ 90% of the Phase-A frontier-as-Core baseline.
- Frontier NEED_MORE (re-entry) rate < 10%.
- Shallow-bias under-prediction rate ≤ 15% (primary production alarm threshold).

14.2.6 Continuous Improvement Loop

- All Core Model calls in production are logged with task_type, structured output, and downstream outcome (Frontier verdict, pipeline success).
- Weekly: escalation cases (frontier NEED_MORE, pipeline FAILED) are collected. The frontier's resolution on these cases becomes new SFT data.
- Monthly: re-run SFT on the accumulated training set. Re-run DPO if under-prediction rate drifts above 15%.
- Per-task metrics (schema validity, template-selection F1) are tracked; any task type degrading beyond its baseline triggers sampling-weight increase in the next training run.

Bootstrap without a trained Core Model
You do not need a trained Core Model to run Engram. Set core_model.provider to any supported API model and the entire system works immediately (at API cost). The fine-tuned local Core Model is an optimization that reduces latency and cost. The recommended path: start with an API provider, collect bootstrap traces from real usage, then train the local model when sufficient traces accumulate.

15. Configuration

15.1 config.yaml

```yaml
# ——— API / Auth ———
api:
  host: "0.0.0.0"
  port: 8000
  api_key: ${Engram_API_KEY}
  rate_limit_query_per_minute: 60
  rate_limit_ingest_per_minute: 600

# ——— Models ———
gating:
  configuration: "dual"             # "dual" | "unified"
  classifier_path: "./models/Engram-gate-v1"
  embedding_model_path: "BAAI/bge-small-en-v1.5"
  classification_threshold: 0.3
  device: "cpu"                     # "cpu" | "cuda"
  max_batch_size: 32
  memory_hit_threshold: 0.75        # §3.1.2 fallback

core_model:
  provider: "local"                 # "local" | "openai" | "anthropic" | "ollama"
  model_path: "Qwen/Qwen3.5-0.8B"
  api_base: null
  api_key: ${CORE_MODEL_API_KEY}
  temperature: 0.1
  max_tokens: 2048
  timeout_seconds: 30

frontier_llm:
  provider: "anthropic"
  model_path: "claude-sonnet-4-20250514"
  api_key: ${FRONTIER_LLM_API_KEY}
  temperature: 0.3
  max_tokens: 4096

# ——— Storage ———
filesystem:
  data_dir: "./data/mem"
  create_dirs: true
  tree_display_max_tokens: 2048

knowledge_graph:
  backend: "neo4j"
  uri: "bolt://localhost:7687"
  writer_username: "Engram_writer"
  writer_password: ${NEO4J_WRITER_PASSWORD}
  reader_username: "Engram_reader"
  reader_password: ${NEO4J_READER_PASSWORD}
  l2_query_timeout_seconds: 5
  l4_query_timeout_seconds: 15
  max_hops: 4

event_ledger:
  backend: "sqlite"
  path: "./data/event_ledger.db"
```

```
  reconciliation_interval_seconds: 60

consolidation:
  db_path: "./data/consolidation.db"
  poll_interval_seconds: 10
  max_concurrent_tasks: 4
  overview_max_tokens: 2048
  overview_debounce_seconds: 30
  manifest_update_delay_seconds: 5
  max_backlog: 10000

session_cache:
  backend: "redis"                # "redis" | "memory" (dev only, single-process)
  redis_url: "redis://localhost:6379"

# ─── Retrieval ───
retrieval:
  l0_skip: false                  # if true, bypass L0 gate entirely
  max_reentries: 2
  max_depth: "L4"
  max_l1_vector_results: 30
  overview_budget_tokens: 6000
  full_doc_budget_tokens: 20000
  msc_compression_threshold: 0.8     # compress MSC when > 80% of token budget

# ─── Decay ───
decay:
  preset: "personal_conversation"    # personal_conversation | coding_agent | knowledge_base
  schedule: "0 3 * * *"
  dormant_floor: 0.05

# ─── Session ───
session:
  timeout_minutes: 30
  window_threshold_ratio: 0.4
  max_turns_before_window: 50
```

15.2 Environment Variables

```
# Required secrets
Engram_API_KEY=...
NEO4J_WRITER_PASSWORD=...
NEO4J_READER_PASSWORD=...
CORE_MODEL_API_KEY=...          # only if core_model.provider != "local"
FRONTIER_LLM_API_KEY=...

# Optional overrides
Engram_CONFIG_PATH=./config.yaml
Engram_DATA_DIR=./data/mem
TREE_DISPLAY_MAX_TOKENS=2048
CONSOLIDATION_POLL_INTERVAL=10
CONSOLIDATION_MAX_CONCURRENT=4
```

16. Appendices

16.1 SQLite Schemas

16.1.1 Event Ledger

```
CREATE TABLE events (
  event_id        TEXT PRIMARY KEY,
  pair_id         TEXT UNIQUE NOT NULL,       -- sha256(session_id || user_idx || assistant_idx)
  session_id      TEXT,
  source          TEXT NOT NULL,              -- 'client' | 'session_compact' | 'manual'
  event_type      TEXT NOT NULL,              -- INGEST | QUERY | SESSION_COMMIT
  payload         TEXT NOT NULL,              -- JSON blob of full turn pair
  status          TEXT NOT NULL DEFAULT 'RECEIVED',
                                              -- RECEIVED | GATED_STORE | GATED_SKIP |
                                              -- INDEXED | COMPLETE | FAILED
  retry_count     INTEGER NOT NULL DEFAULT 0,
  error_message   TEXT,
  created_at      TEXT NOT NULL DEFAULT (datetime('now')),
  processed_at    TEXT
);

CREATE INDEX idx_events_status ON events(status, created_at);
CREATE INDEX idx_events_session ON events(session_id);
```

16.1.2 Outbox (filesystem → KG handoff)

```
CREATE TABLE fs_outbox (
  event_id      TEXT PRIMARY KEY,
  source_uri    TEXT NOT NULL,
  state         TEXT NOT NULL,                -- WRITTEN | INDEXED | INDEX_FAILED
  retry_count   INTEGER NOT NULL DEFAULT 0,
  written_at    TEXT NOT NULL,
  last_attempt  TEXT,
  error_message TEXT
);

CREATE INDEX idx_fs_outbox_state ON fs_outbox(state, written_at);
```

16.1.3 Extractions

```
CREATE TABLE extractions (
  event_id       TEXT PRIMARY KEY,
  resolved_text  TEXT NOT NULL,
  triplets       TEXT NOT NULL,               -- JSON array
  l0_abstract    TEXT NOT NULL,
  created_at     TEXT NOT NULL DEFAULT (datetime('now'))
);
```

16.1.4 Linked Entities

```
CREATE TABLE linked_entities (
  event_id      TEXT NOT NULL,
  triplet_idx   INTEGER NOT NULL,             -- index into extractions.triplets
  subject_node_id TEXT,
  object_node_id  TEXT,
  PRIMARY KEY (event_id, triplet_idx)
);
```

16.1.5 Consolidation Tasks

See §7.2 for the full schema.

16.2 Prompt File Index

```
prompts/
├── gate_write.j2          # §8.2 — write-path gate
├── extract.j2             # §5.4.3 — S-R-O + L0 abstract
├── l1_plan.j2             # §3.2 — L1 plan (+ mode, entry points, vector queries)
├── ln_plan.j2             # §3.3–§3.5 — L2/L3/L4 plan with fused judgment
├── dedup.j2               # §5.4.6 — dedup / conflict classification
├── entity_link.j2         # §5.4.4 — entity linking
├── overview.j2            # §7.3 — directory overview generation
├── session_compact.j2     # §8.3 — session compaction
├── unmerge.j2             # §8.6 — unmerge operation
└── relations.yaml         # §6.4.2 — controlled vocabulary of relation labels
```

16.3 Cypher Template Files

```
templates/cypher/
├── t_children_of.cypher
├── t_neighbours_by_relation.cypher
├── t_path_between.cypher
├── t_temporal_filter.cypher
├── t_history_chain.cypher
├── t_find_by_uri_prefix.cypher
├── t_cross_references.cypher
└── t_top_k_vector.cypher
```

Each template is validated at startup against the Engram_reader role's permission set. Any template that would require write privileges fails the startup check.

16.4 Glossary

Term	Definition
MSC	Minimal Sufficient Context. The principle that the context delivered to the frontier LLM should contain exactly enough information to answer the query correctly, and no more.
L0 / L1 / L2 / L3 / L4	Retrieval cascade levels. See §3.
Mode (AGFS / KG / HYBRID)	L1 planner's choice of traversal strategy. See §3.2.2.
Plan / Execute / Judge	The three phases at each level. See §3.

Term	Definition
Fused plan output	Single Core Model call at L2+ that combines previous-level judgment with next-level plan. See §4.2.
Retrieval Orchestrator	The top-level component that owns the read loop. See §4.
Frontier re-entry	A NEED_MORE verdict from the Frontier LLM that causes the orchestrator to re-run the cascade at a deeper level. See §4.3.
mem:// URI	Canonical identifier and filesystem locator for a memory. See §6.1.
source_uri	The mem:// URI stored on every KG node; unique and immutable.
Outbox	SQLite tables (fs_outbox) that track cross-store handoff state for crash recovery. See §5.1.
Pair / Turn group	The unit of ingest: a user turn + assistant turn (pair), or an extended sequence including tool calls (turn group). See §5.2.
Session compaction	Core Model task that restructures and shortens older session turns when the session window threshold is reached. See §8.3.
Coverage (aspects)	Natural-language lists of answered/unanswered claims used in place of a numeric ERV formula. See §4.5.
Dual vs unified gating	Whether the L0 classifier and the embedding generator are separate models (dual, default) or share a backbone (unified). See §1.5.